

```
import socket
import base64
import os
from ar_mess import ar_directory
import threading
from pathlib import Path
from main_menu import mange
import project_db
import time
from ar_mess import DatabaseManager as d_manager

SERVER_HOST = 'localhost'
SERVER_PORT = 5002
BUFFER_SIZE = 4096

class CloudClient:
    def __init__(self, host=SERVER_HOST, port=SERVER_PORT):
        self.host = host
        self.port = port
        self.sock = None
        self.username = None

    def connect(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.connect((self.host, self.port))

    def disconnect(self):
        if self.sock:
            self.sock.close()
            self.sock = None

    def send_and_receive(self, message):
        self.sock.send(message.encode())
        return self.sock.recv(BUFFER_SIZE).decode()

    def login(self): #fix in servear
        self.connect()
```

```

username = input("Username: ")
password = input("Password: ")
response = self.send_and_receive(f"LOGIN[{username}][{password}]")
print("Server:", response)
if response == "LOGIN_SUCCESS":
    self.username = username
    self.password = password
    return True
self.disconnect()
return False

def check_text(self, text):
    allowed_chars =
    set("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_")
    if len(text) == 0:
        return False
    for char in text:
        if char not in allowed_chars:
            return False
    return True

#fix max length of username and password and password

def signup(self): #need to add email
    #self.connect()
    username = input("Choose username: ")
    password = input("Choose password: ")
    email = input("Enter your email: ")
    if self.check_text(username) == False or self.check_text(password) == False:
        print("Username or password cant be use, pls try again.")
    self.signup()
    response = self.send_and_receive(f"SIGNUP[{username}][{password}][{email}]")
    print("Server:", response)
    if response == "SIGNUP_SUCCESS":
        self.username = username

```

```

self.password = password

return True

self.disconnect()

return False

#C:\Data\roy\school\cyber\cloud\copy

def project_directory(self):

p1 = d_manager("project_data.db")

self.db = p1

p1.print_all_projects()

self.project_name = input("Enter project name or type new to create a new project: ").strip()

if self.project_name == "new":

self.project_name = input("Enter project name: ").strip()

friend = "y" #input("do you have an ip already? (y/n) ").strip().lower()

if friend == "y":

self.host = "127.0.0.1" #input("Enter the ip: ").strip()

space = 1 #int(input("Enter the space you want to rent in GB: ").strip())

code = 0 #int(input("Do you have a code? (Write code for yes, 0 for no): ").strip())

path = "C:\\Data\\roy\\school\\cyber\\cloud\\copy" #input("Enter the path of the project: ").strip()

ok = True

if code != 0 and ok:

response = self.send_and_receive(f"CREATE_PROJECT|{self.project_name}|{space}|0|{code}")

if self.signup() == False:

print("Signup failed, try again.")

self.project_directory()

print("Server:", response)

if response == "PROJECT_CREATED":

p1.add_project(self.project_name, self.username, self.host, 0,0, space, path)

else:

print("-----")

self.project_directory()

else:

```

```

price = 1 #int(input("Enter the price you want to pay for 1 GB per month: ").strip())

response = self.send_and_receive(f"CREATE_PROJECT|{self.project_name}|{space}|{price}|0")

if self.signup() == False:

print("Signup failed, try again.")

self.project_directory()

#print("Server:", response)

#if response == "PROJECT_CREATED":

else:

p1.add_project(self.project_name, self.username, self.host, price,0, space, path)

self.project_directory = path

else:

#rent a server from the market

pass

else:

ip = p1.get_ip_by_project(self.project_name)

if ip is not None:

self.host = ip

print(f'entring '{self.project_name}'.')

self.project_directory = p1.get_project_path(self.project_name)

self.connect() #need to continue

username = input("Username: ")

password = input("Password: ")

response = self.send_and_receive(f"LOGIN|{username}|{password}")

if response == "LOGIN_SUCCESS":

self.username = username

self.password = password

print(f'Successfully logged in to project '{self.project_name}' at {self.host}.')

print("initialzing project")

response = self.send_and_receive(f"OPEN_PROJECT|{self.project_name}")

if response == "PROJECT_OPENED":

print(f'Successfully connected to project '{self.project_name}' at {self.host}.')

```

```

else:
    print(f"Failed to open project '{self.project_name}': {response}")
    self.disconnect()

return

else:
    print(f"Failed to log in to project '{self.project_name}': {response}")
    self.disconnect()

    self.project_directory()

else:
    print("Project not found.")
    self.project_directory()

"""

def project_directory_2(self):
    #project_db.create_json_file()
    p1 = project_db.TextFileManager()
    response = self.send_and_receive("PROJECT_LIST")
    print("Server:", response)

    self.project_name = input("Enter project name or type New to create a new project: ").strip()
    if self.project_name.lower() == "new":
        self.project_name = input("Enter project name: ").strip()
        self.project_type = 1
        response = self.send_and_receive(f"CREATE_PROJECT[{self.project_type}]{self.project_name}")
        print("Server:", response)
        if response == "PROJECT_CREATED":
            self.project_directory = input("Enter project directory path: ").strip()
            if not os.path.exists(self.project_directory):
                print("Directory not found.")
            while True:
                self.project_directory = Path(input("Enter project directory path: ").strip())
                if os.path.exists(self.project_directory):
                    break

```

```

p1.add_project(self.project_name, self.project_directory, self.project_type)

x = ar_directory(Path(self.project_directory))

files_data = x.return_paths()

lis = [data["path"] for data in files_data]

self.upload_all_files(lis)

print(f"Project '{self.project_name}' created successfully.")

else:

print("Failed to create project.")

else:

response = self.send_and_receive(f"OPEN_PROJECT[{self.project_name}]")

print("Server:", response)

if response == "PROJECT_NOT_FOUND":

print("Project not found.")

self.project_directory()

else:

print(f"entring '{self.project_name}'.")

self.project_directory = p1.get_project_path(self.project_name)

"""

def upload_file(self,path):

try:

#filename = os.path.basename(path)

filename = os.path.basename(path)

with open(path, "rb") as f:

file_bytes = f.read()

b64 = base64.b64encode(file_bytes).decode()

# Create a new socket for each upload to avoid threading conflicts

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as upload_sock:

upload_sock.connect((self.host, self.port))

# First login again to establish identity

upload_sock.send(f"LOGIN[{self.username}]{self.password}".encode())

response = upload_sock.recv(BUFFER_SIZE).decode().strip()

```

```

if response != "LOGIN_SUCCESS":
    raise Exception(f"Authentication failed: {response}")

# Send project context
current_project = os.path.basename(self.project_directory)
upload_sock.send(f"OPEN_PROJECT|{self.project_name}".encode())
response = upload_sock.recv(BUFFER_SIZE).decode().strip()

if response != "PROJECT_OPENED":
    raise Exception(f"Failed to open project context: {response}")

#send upload command
header = f"UPLOAD|{filename}|{len(b64)}"
upload_sock.sendall((header).encode())
response = upload_sock.recv(BUFFER_SIZE).decode().strip()

if response == "READY_TO_RECEIVE":
    upload_sock.sendall(b64.encode())
    response = upload_sock.recv(BUFFER_SIZE).decode().strip()

if response != "UPLOAD_SUCCESS":
    raise Exception(f"Upload failed: {response}")

print(f"[{threading.current_thread().name}] Uploaded: {filename}")

db = d_manager("project_data.db")
db.add_file(self.project_name, filename, len(file_bytes), time.time())

except Exception as e:
    print(f"[ERROR] Failed to upload {path}: {e}")

def upload_all_files(self, file_paths, max_threads=5):
    semaphore = threading.Semaphore(max_threads)
    threads = []

    def thread_upload(path):
        with semaphore:
            try:
                self.upload_file(path)
            except Exception as e:
                print(f"[ERROR] Failed to upload {path}: {e}")

```

```

for path in file_paths:

    t = threading.Thread(target=thread_upload, args=(path,))

    threads.append(t)

    t.start()

for t in threads:

    t.join()

print("■ All uploads finished.")

def download_file(self, filename):

    try:

        # Create a new socket for each download to avoid threading conflicts
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as download_sock:

            download_sock.connect((self.host, self.port))

            # First login again to establish identity
            download_sock.send(f"LOGIN[{self.username}]{self.password}".encode())

            response = download_sock.recv(BUFFER_SIZE).decode().strip()

            if response != "LOGIN_SUCCESS":

                raise Exception(f"Authentication failed: {response}")

            # Send project context
            download_sock.send(f"OPEN_PROJECT[{self.project_name}].encode())

            response = download_sock.recv(BUFFER_SIZE).decode().strip()

            if response != "PROJECT_OPENED":

                raise Exception(f"Failed to open project context: {response}")

            # Send download command
            download_sock.send(f"DOWNLOAD[{filename}].encode())

            file_data = download_sock.recv(BUFFER_SIZE)

            if file_data:

                with open(filename, "wb") as f:

                    f.write(file_data)

                print(f"Downloaded: {filename}")

            else:

                print(f"Failed to download: {filename}")

```



```

except Exception as e:

print(f"[ERROR] Failed to download {filename}: {e}")

def download_project(self):

msg = f"DOWNLOAD_PROJECT[{self.project_name}]"

response = self.send_and_receive(msg)

print("Server:", response)

if response.startswith("DOWNLOAD"):

_, size_str = response.split("|")

expected_size = int(size_str)

self.sock.send(b"READY")

received_data = b""

while len(received_data) < expected_size:

chunk = self.sock.recv(BUFFER_SIZE)

if not chunk:

break

received_data += chunk

if len(received_data) == expected_size:

zip_data = base64.b64decode(received_data)

with open(f"{self.project_name}.zip", "wb") as f:

f.write(zip_data)

print(f"Project '{self.project_name}' downloaded successfully.")

else:

print(f"Failed to download project: Expected {expected_size} bytes, received {len(received_data)} bytes.")

def run(self):

self.connect()

"""

while True:

where = input("do you want to manage your server or your project? (manage/projects) ").lower()

if where == "manage":

m1 = mange()

```

```

m1.choose()

break

continue

break

#need to first gives a list of projects

action = input("Login or Signup? ").lower()

if action == "login" and self.login():

break

elif action == "signup" and self.signup():

break

else:

print("Try again.")

"""

print(f"Welcome, {self.username}!")

self.project_directory()

while True:

cmd = input("Type 'upload' to send a file, 'upload all', 'download file', 'download all' or 'quit': ").lower()

if cmd == "upload":

path = input("Enter full path to file: ")

self.upload_file(path)

elif cmd == "upload all":

#d1 = ar_directory(self.project_name, Path(self.project_directory))

#d1.print_all_records()

#files = d1.run()

#file_paths = [file['path'] for file in files]

#self.upload_all_files(file_paths, d1)

files = self.db.run(self.project_name)

file_paths = [file['path'] for file in files]

self.upload_all_files(file_paths)

print("uploading finished.")

#d1.print_all_records()

```

```
elif cmd == "download":  
    filename = input("Enter filename to download: ")  
    self.download_file(filename)  
elif cmd == "download all":  
    self.download_project()  
elif cmd == "quit":  
    break  
else:  
    print("Unknown command.")  
    self.disconnect()  
if __name__ == "__main__":  
    client = CloudClient()  
    client.run()
```